Informe TPFI Ingenieria de software 2

Trabajo Práctico Final Integrador - Ingeniería de Software II (UADER-FCyT). Grupo: Balbuena Juan, Romero Gonzalo, Romero Valentin, Sangalli Luciano, Zaragoza Nahuel.

Repositorio utilizado: https://github.com/gonzafc/tpFinalIS2

1. Entorno de desarrollo

Se utilizó Python 3.10 o superior con un entorno virtual (venv). Las dependencias están en requirements.txt: boto3 para DynamoDB y las herramientas de prueba y calidad. El acceso a AWS se configuró con el AWS CLI (aws configure) usando las credenciales de la cátedra, que no forman parte del repositorio.

Comandos a ejecutar:

```
cd $HOME\Desktop\tpfinal_software
venv\Scripts\Activate.ps1
python --version
pip install -r requirements.txt
pip list | findstr "boto3 pytest black ruff mypy pyright bandit pdoc"
```

2. Estructura del proyecto

El esqueleto se generó con CookieCutter (cookiecutter-pypackage) y se adoptó su andamiaje: licencia MIT, plantillas de issues y de pull request, .editorconfig, SECURITY.md y CODE_OF_CONDUCT.md. Se mantuvo un layout plano porque el enunciado exige black, ruff, mypy, pyright, bandit y pdoc, y el CI ya los ejecuta así (detalle en docs/cookiecutter.md).

El código está en el paquete tpfi: config, protocol, net, singleton, observer, proxy, server, clients y los módulos de acceso a tablas (db). Los tres programas están en la raíz y los tests en la carpeta tests.

Comandos a ejecutar:

```
# Generación del esqueleto
cookiecutter gh:audreyfeldroy/cookiecutter-pypackage

# Estructura del repositorio
cd $HOME\Desktop\tpfinal_software
dir
git ls-files
type docs\cookiecutter.md
```

3. Tablas de DynamoDB

El sistema usa dos tablas ya existentes en la cuenta de AWS de la cátedra. CorporateData guarda los datos corporativos (id, cp, CUIT, domicilio, idreq, idSeq, localidad, provincia, sede, seqID, telefono, web). CorporateLog guarda la pista de auditoría: cada acción del servidor genera un registro con UUID, sessionid, action, timestamp, CPUid y datos del equipo.

Comandos a ejecutar:

```
aws dynamodb list-tables
aws dynamodb scan --table-name CorporateData
aws dynamodb scan --table-name CorporateLog --max-items 5
```

También se pueden mostrar las tablas desde la consola de AWS (DynamoDB, Explore table items).

4. Patrón Singleton

La clase SingletonMeta (tpfi/singleton.py) es una metaclase con un lock, por lo que garantiza una única instancia por clase aun si se crea desde varios hilos. CorporateDataDB y CorporateLogDB (carpeta tpfi/db) la usan, de modo que hay un único punto de acceso físico a cada tabla.

Comandos a ejecutar:

```
type tpfi\singleton.py
type tpfi\db\data_table.py
pytest tests/test_singleton.py -v --no-cov
```

```
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> type tpfi\singleton.py


class SingletonMeta(type):
    """Metaclase que garantiza una Ãºnica instancia por clase."""

    _instances: ClassVar[dict[type, Any]] = {}
    _lock: ClassVar[threading.Lock] = threading.Lock()

    def __call__(cls, *args: Any, **kwargs: Any) -> Any:
        """Devuelve la instancia Ãºnica, creÃ¡ndola si no existe."""
        with SingletonMeta._lock:
            if cls not in SingletonMeta._instances:
                SingletonMeta._instances[cls] = super().__call__(*args, **kwa
rgs)
            return SingletonMeta._instances[cls]

    @classmethod
    def reset(mcs, cls: type | None = None) -> None:
        """Descarta instancias (Ãºtil en tests)."""
        with mcs._lock:
            if cls is None:
                mcs._instances.clear()
            else:
                mcs._instances.pop(cls, None)
```

```
Problems  Output  Debug Console  Terminal  Ports

(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> pytest
tpfi\apps\observerclient.py              23      0   100%
tpfi\apps\singletonclient.py             47      0   100%
tpfi\apps\singletonproxyobserver.py      35      0   100%
tpfi\cliargs.py                          26      1    96%   29
tpfi\clients.py                          92      2    98%   47, 124
tpfi\config.py                           17      0   100%
tpfi\db\__init__.py                       0      0   100%
tpfi\db\data_table.py                    47      3    94%   27-30
tpfi\db\keys.py                          12      0   100%
tpfi\db\log_table.py                     33      3    91%   30-33
tpfi\logging_setup.py                    12      0   100%
tpfi\net.py                              37      0   100%
tpfi\observer.py                         66      0   100%
tpfi\protocol.py                         49      0   100%
tpfi\proxy.py                            62      0   100%
tpfi\server.py                           99      6    94%   56, 123-124, 133,
 143-144
tpfi\service.py                           7      0   100%
tpfi\singleton.py                        17      0   100%
------------------------------------------------------------------
TOTAL                                   682     15    98%
Required test coverage of 85% reached. Total coverage: 97.80%
========================== 154 passed in 28.00s ==========================
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software>
```


5. Patrón Observer

En tpfi/observer.py, Observer es una clase abstracta (update y close), Subject

mantiene la lista de suscriptores con attach, detach y notify, y SubscriptionManager registra cada socket como SocketObserver. Cuando un suscriptor deja de responder, notify lo elimina y cierra su conexión.

Comandos a ejecutar:

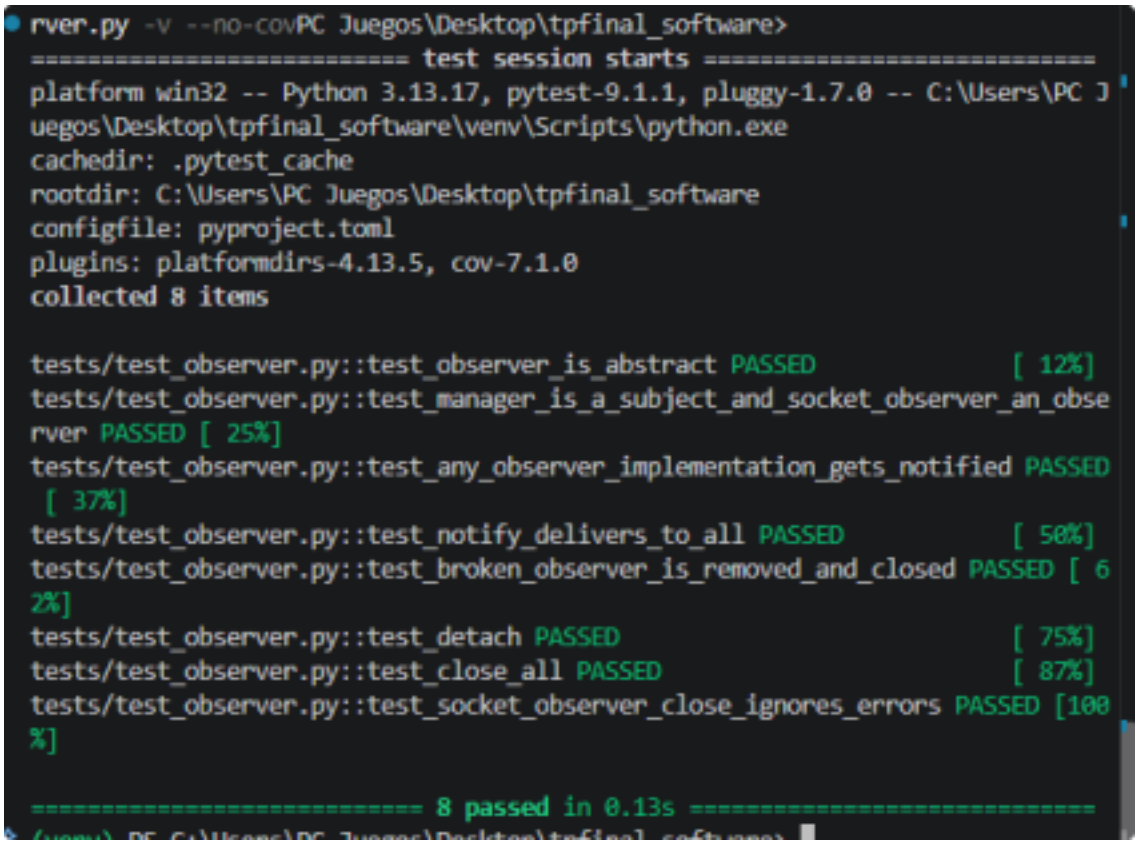
type tpfi\observer.py
pytest tests/test_observer.py -v --no-cov

```
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> type tpfi\observer.py
    def notify(self, message: Any) -> int:
        """Notifica a todos y elimina los caÃdos; devuelve los entregados."""
        with self._lock:
            observers = list(self._observers)
        delivered = 0
        for observer in observers:
            if observer.update(message):
                delivered += 1
            else:
                self.detach(observer)
                observer.close()
        return delivered

    def close_all(self) -> None:
        """Cierra y elimina a todos los suscriptores."""
        with self._lock:
            observers, self._observers = self._observers, []
        for observer in observers:
            observer.close()


class SubscriptionManager(Subject):
    """Gestor de suscripciones del servidor."""

    def subscribe(self, uuid: str, sock: socket.socket) -> SocketObserver:
        """Registra un socket como suscriptor."""
        observer = SocketObserver(uuid, sock)
        self.attach(observer)
        return observer
```



6. Patrón Proxy

CorporateDataProxy (tpfi/proxy.py) y CorporateDataDB implementan la misma interfaz DataService (tpfi/service.py). El proxy registra cada acción en CorporateLog antes de delegar en el sujeto real y, después de un set, notifica a los observers. Si falla la auditoría, no se modifica ningún dato.

Comandos a ejecutar:

```
type tpfi\proxy.py
type tpfi\service.py
pytest tests/test_proxy.py -v --no-cov
```

```
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> type tpfi\proxy.py
>>
        return self._subject.get(key)

    def set(
        self,
        key: str,
        fields: dict[str, Any],
        *,
        client: str = "unknown",
        session: str = "",
    ) -> dict[str, Any]:
        """Audita, modifica un registro y notifica a los suscriptos."""
        self._audit(client, session, "set", key)
        item = self._subject.set(key, fields)
        delivered = self._subs.notify(item)
        logger.debug("set %s notificado a %d observers", key, delivered)
        return item

    def subscribe(self, *, client: str = "unknown", session: str = "") -> Non
e:
        """Audita una suscripciÃ³n (el socket lo registra el servidor)."""
        self._audit(client, session,

    def list(
        self, *, client: str = "unknown", session: str = ""
    ) -> list[dict[str, Any]]:
        """Audita y devuelve todos los registros."""
        self._audit(client, session, "list")
        return self._subject.list()
```

```
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> type tpfi\service.py
"""Interfaz comÃºn del sujeto real y del proxy (patrÃ³n Proxy)."""

from __future__ import annotations

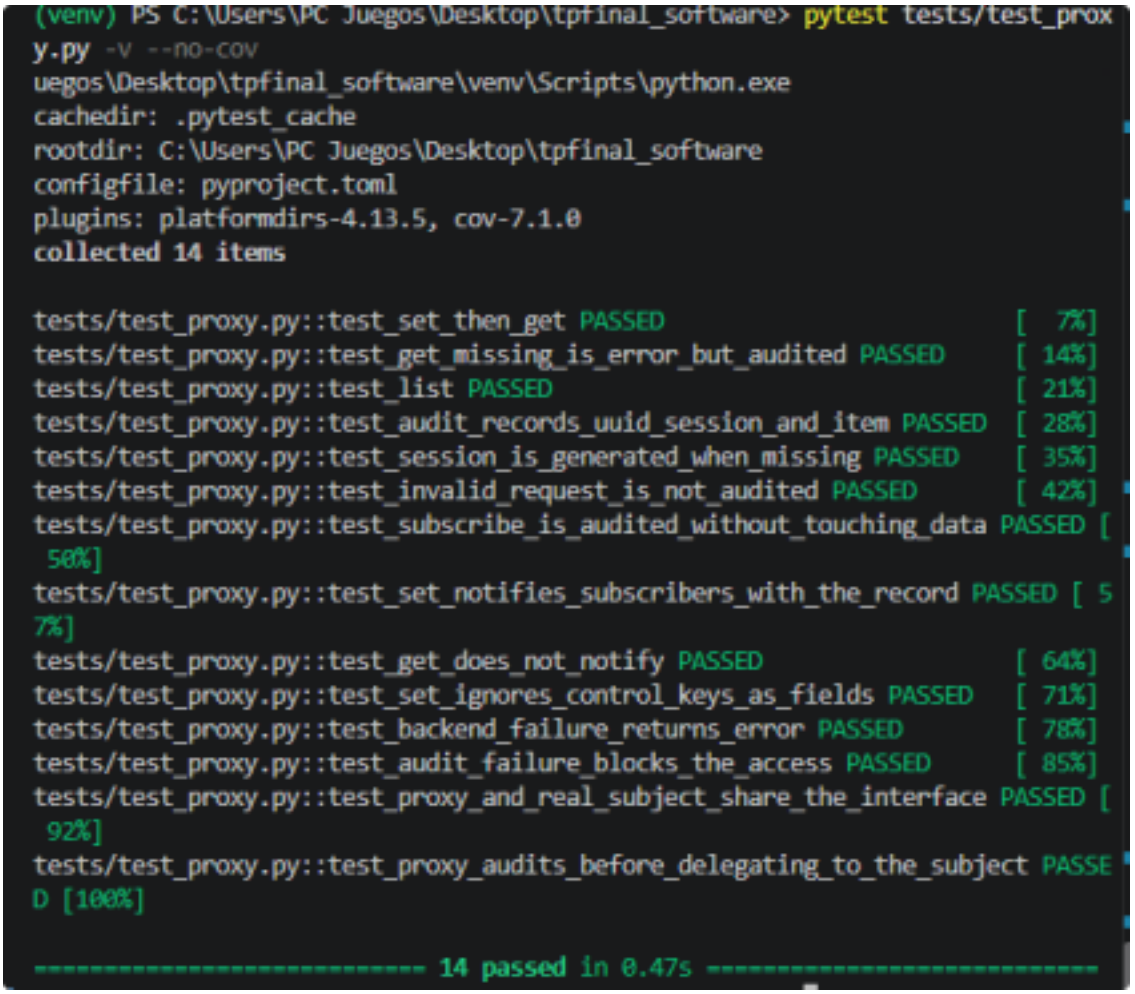
from typing import Any, Protocol, runtime_checkable


@runtime_checkable
class DataService(Protocol):
    """Operaciones sobre CorporateData que comparten el sujeto real y el prox
y."""

    def get(self, key: str) -> dict[str, Any] | None:
        """Recupera un registro o ``None`` si no existe."""
        ...

    def set(self, key: str, fields: dict[str, Any]) -> dict[str, Any]:
        """Modifica o crea un registro y lo devuelve."""
        ...

    def list(self) -> list[dict[str, Any]]:
        """Devuelve todos los registros."""
```



7. Servidor y clientes en funcionamiento

Se ejecutaron los tres programas a la vez en tres terminales. El servidor (singletonproxyobserver.py, puerto 8080 por defecto) atiende cada conexión en un hilo. singletonclient.py envía un pedido get, set o list desde un archivo JSON. observerclient.py queda suscripto y muestra cada registro modificado; si el servidor no está, reintenta cada 30 segundos.

Comandos a ejecutar:

```
# Preparar los archivos de pedido (Terminal 3)
Set-Content set1.json '{"UUID": "123456789", "ACTION": "set", "ID": "UADER FCyT-IS2", "cp":
"3260", "sede": "FCyT"}'
Set-Content set3.json '{"UUID": "123456789", "ACTION": "set", "ID": "UADER FCyT-IS2", "telefono":
"03442 999999", "domicilio": "Calle Nueva 123"}' Set-Content get1.json '{"UUID": "123456789",
"ACTION": "get", "ID": "UADER FCyT-IS2"}'
Set-Content get2.json '{"UUID": "123456789", "ACTION": "get", "ID": "NO EXISTE"}'
Set-Content list1.json '{"UUID": "123456789", "ACTION": "list"}' Set-Content malo1.json
'{"ACTION": "list"}'
```

```
# Terminal 1: servidor
python singletonproxyobserver.py -v

# Terminal 2: observer
python observerclient.py -v

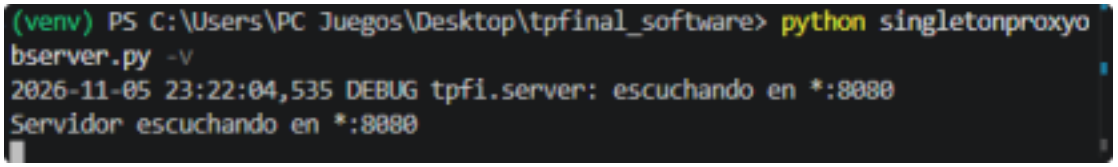
# Terminal 3: cliente
python singletonclient.py "-i=set1.json"
python singletonclient.py "-i=get1.json"
python singletonclient.py "-i=list1.json"

# Modificar los datos de un cliente
python singletonclient.py "-i=set3.json"
python singletonclient.py "-i=get1.json"

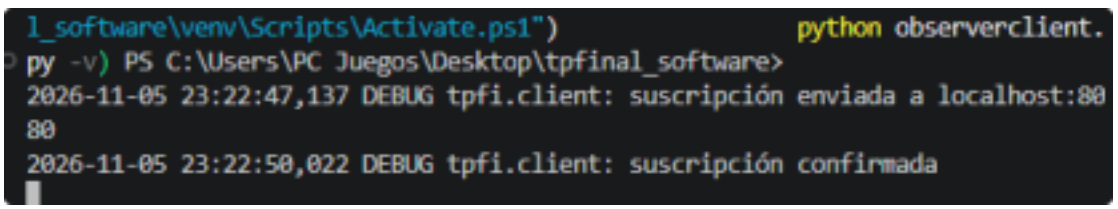
# Errores
python singletonclient.py "-i=get2.json"
echo $LASTEXITCODE
python singletonclient.py "-i=malo1.json"
python singletonclient.py
python singletonclient.py "-i=get1.json" "-p=abc"
python observerclient.py "-p=abc"
python singletonproxyobserver.py "-p=99999"

# Servidor caído y segundo servidor en el mismo puerto
python singletonclient.py "-i=get1.json" "-p=9999"
python observerclient.py "-p=9999" "-r=5"
python singletonproxyobserver.py
```

Terminal 1:



Terminal 2:



terminal 3: Todos los test

8. Auditoría en CorporateLog

Cada pedido válido deja un registro en CorporateLog con el UUID del cliente, el identificador de sesión, la acción, el ID consultado, la fecha y hora en UTC, el CPUid y los datos del equipo. Los pedidos sin los datos mínimos se rechazan y no generan registro.

Comandos a ejecutar:

```
python referencia\ver_log.py
```

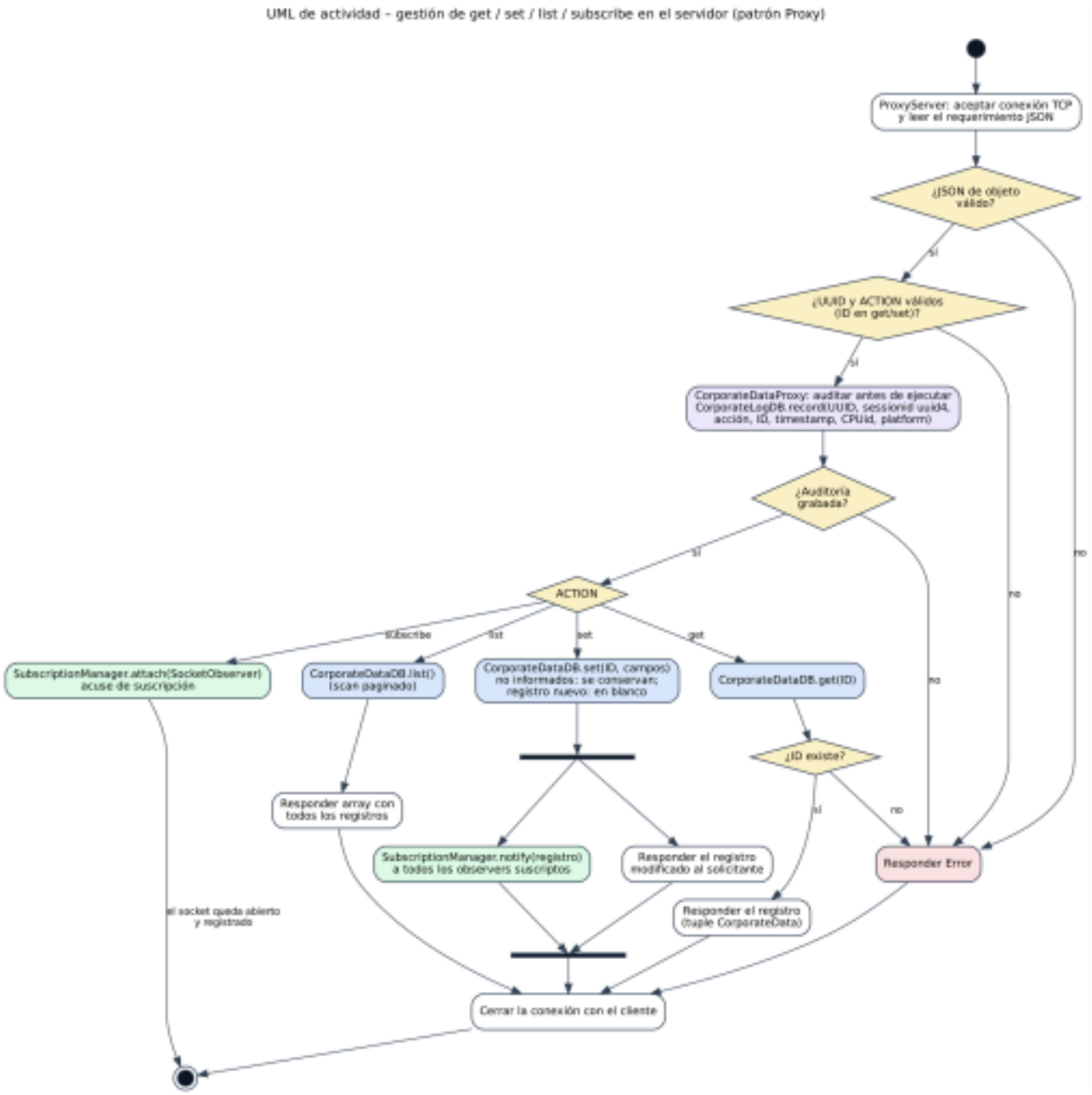


9. Diagramas UML

Se elaboraron dos diagramas con Graphviz: un diagrama de actividad que describe cómo el servidor procesa un pedido a través del proxy (validación, auditoría, get, set, list y subscribe) y un diagrama de estado del observer (estados de conexión, suscripción, recepción de notificaciones y reintento).

Comandos a ejecutar:

```
start docs\uml\proxy_activity.png
start docs\uml\observer_state.png
```



10. Pruebas automáticas

Se escribieron 154 tests con pytest. Los tests unitarios y de integración usan tablas DynamoDB simuladas en memoria y sockets reales en localhost. Los tests de aceptación (tests/test_acceptance.py) ejecutan los tres programas como procesos reales y verifican

el camino feliz con las entradas de CorporateLog, los argumentos mal formados de cada programa, los pedidos sin datos mínimos, el cliente con el servidor caído y el doble arranque del servidor. La cobertura mínima exigida es 85 %.

Comandos a ejecutar:

pytest -v
pytest tests/test_acceptance.py -k happy_path -v --no-cov
pytest tests/test_acceptance.py -v --no-cov

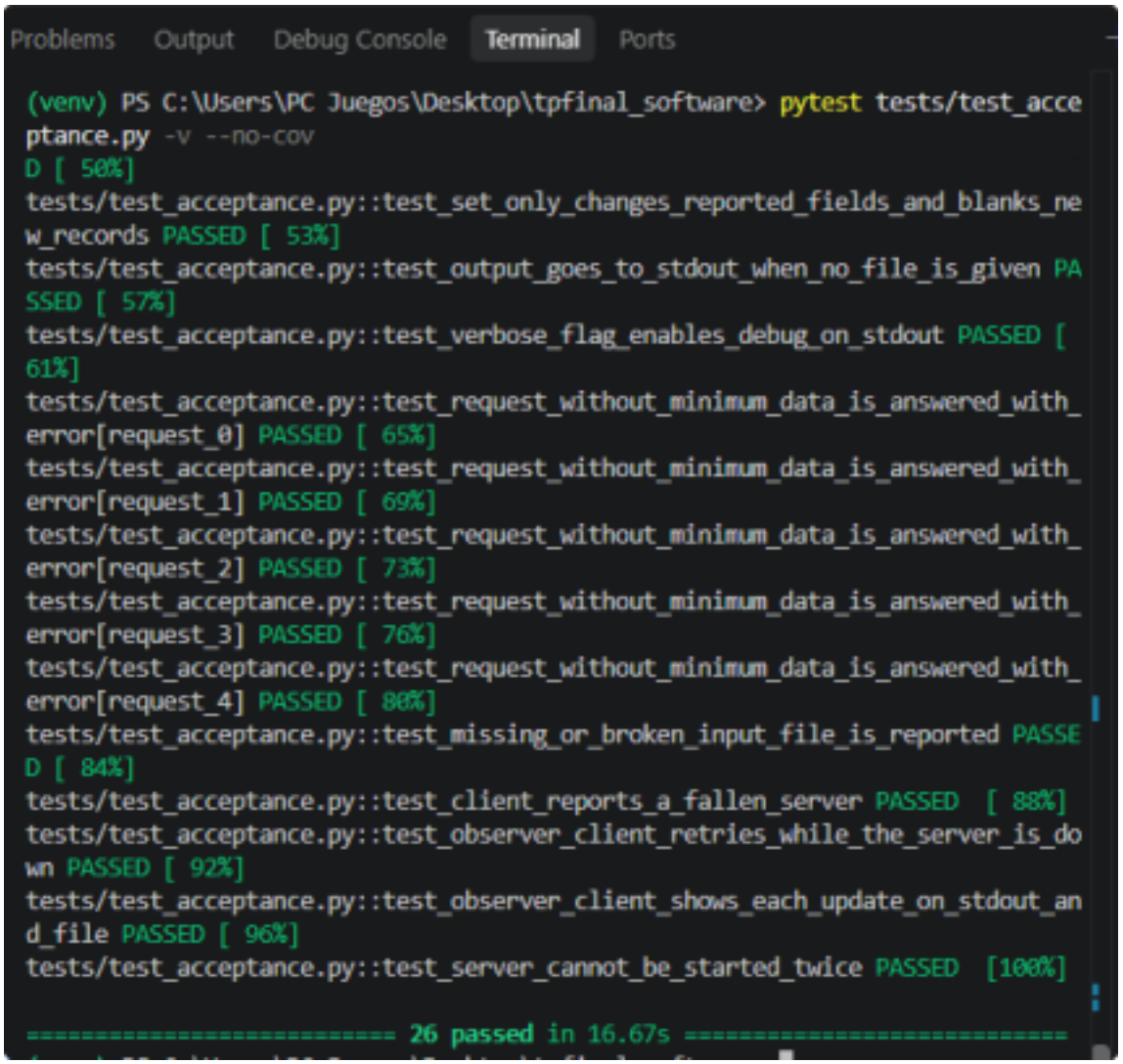
```
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software> pytest -v

Name                                      Stmts   Miss  Cover   Missing
-----------------------------------------------------------------------
tpfi\__init__.py                              1      0   100%
tpfi\apps\__init__.py                         0      0   100%
tpfi\apps\observerclient.py                  23      0   100%
tpfi\apps\singletonclient.py                 47      0   100%
tpfi\apps\singletonproxyobserver.py          35      0   100%
tpfi\cliargs.py                              26      1    96%   29
tpfi\clients.py                              92      2    98%   47, 124
tpfi\config.py                               17      0   100%
tpfi\db\__init__.py                           0      0   100%
tpfi\db\data_table.py                        47      3    94%   27-30
tpfi\db\keys.py                              12      0   100%
tpfi\db\log_table.py                         33      3    91%   30-33
tpfi\logging_setup.py                        12      0   100%
tpfi\net.py                                  37      0   100%
tpfi\observer.py                             66      0   100%
tpfi\protocol.py                             49      0   100%
tpfi\proxy.py                                62      0   100%
tpfi\server.py                               99      6    94%   56, 123-124, 133,
 143-144
tpfi\service.py                               7      0   100%
tpfi\singleton.py                            17      0   100%
-----------------------------------------------------------------------
TOTAL                                       682     15    98%
Required test coverage of 85% reached. Total coverage: 97.80%
============================ 154 passed in 26.07s =============================
(venv) PS C:\Users\PC Juegos\Desktop\tpfinal_software>
```

```
ptance.py -k happy_path -v --no-cov
============================= test session starts =============================
platform win32 -- Python 3.13.17, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\PC J
uegos\Desktop\tpfinal_software\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\PC Juegos\Desktop\tpfinal_software
configfile: pyproject.toml
plugins: platformdirs-4.13.5, cov-7.1.0
collected 26 items / 25 deselected / 1 selected

tests/test_acceptance.py::test_happy_path_set_get_list_with_audit_trail PASSE
D [100%]

======================= 1 passed, 25 deselected in 1.03s ======================
```



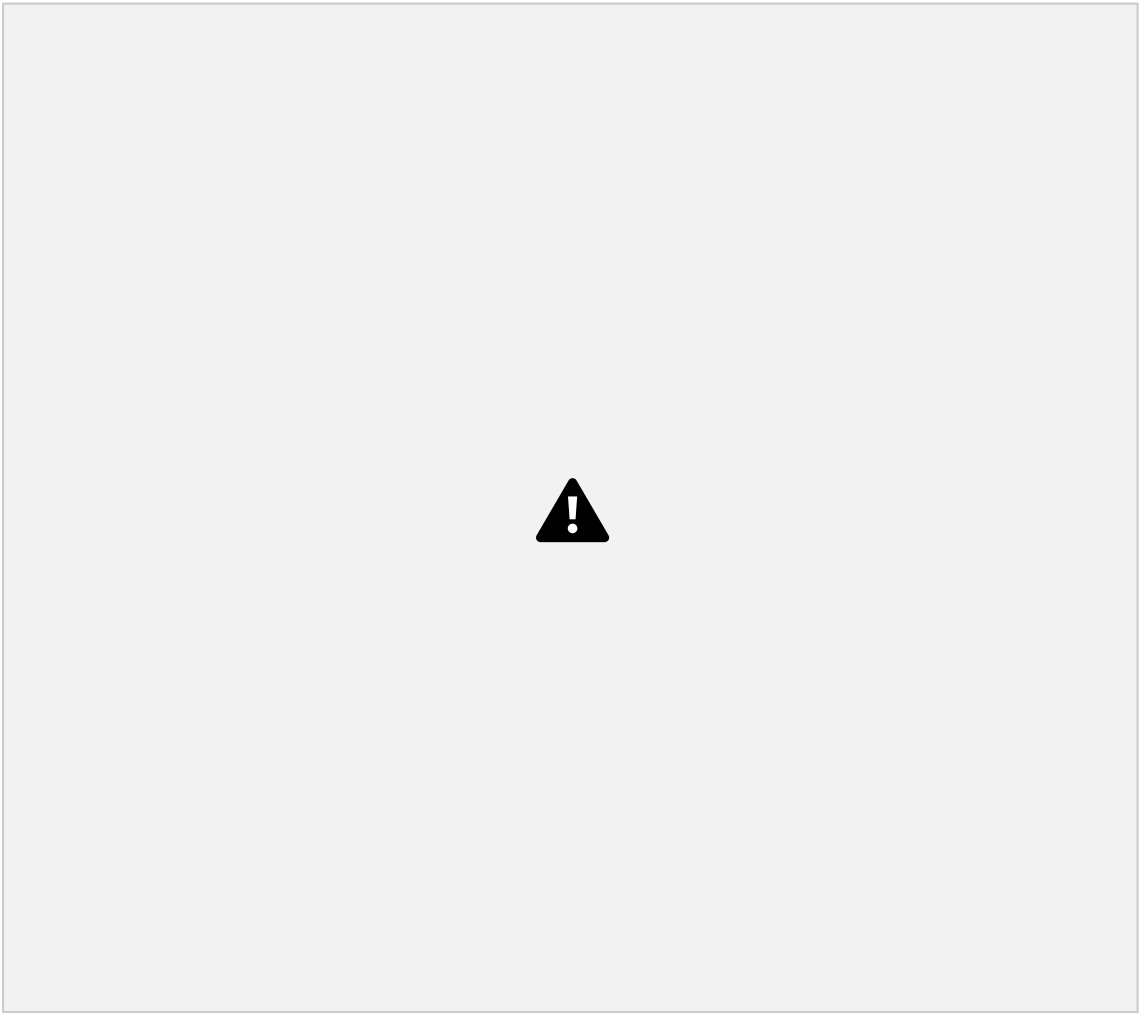
11. Calidad del código

Se aplicaron las herramientas pedidas: black (formato), ruff (estilo PEP 8 y PEP 257), mypy y pyright (tipos) y bandit (seguridad). Todas se ejecutan sin errores. La configuración está en pyproject.toml.

Comandos a ejecutar:

```
black --check .
ruff check .
mypy
pyright
bandit -r tpfi -c pyproject.toml
```





12. Documentación y CI/CD

El repositorio incluye README.md (uso, protocolo, arquitectura), CHANGELOG.md, CONTEXT.md (registro del uso de IA y estado de cada consigna), VERSION (0.2.0), BUILD, LICENSE (MIT) y requirements.txt. La documentación de la API se genera con pdoc (pdoc tpfi -o docs/api). El workflow de GitHub Actions (.github/workflows/ci.yml) ejecuta en cada push ruff, black, mypy, pyright, pytest con cobertura, bandit y pdoc.

Comandos a ejecutar:

```
pdoc tpfi -o docs/api
start docs\api\index.html
type CHANGELOG.md
type CONTEXT.md
```

13. Uso de IA y conclusión

Se utilizó Claude (Anthropic) como asistente para el diseño, la generación de código y ayuda en varias situaciones.

Se cumplieron las consignas del enunciado: los tres programas, los patrones Singleton, Observer y Proxy, la auditoría en CorporateLog, los diagramas UML, las pruebas con cobertura superior al 85 %, las herramientas de calidad y la integración continua. Quedó como opcional, sin realizar, el análisis con Trufflehog; en su lugar se controló manualmente que no se subieran credenciales.